1. Contextualización del problema

Durante este ejercicio demostrare lo aprendido sobre redes neuronales usando PyTorch, una herramienta que permite crear y entrenar modelos con bastante flexibilidad y claridad en su estructura. A diferencia de otras plataformas como TensorFlow, PyTorch nos permite ver paso a paso cómo se comporta el modelo, entender mejor su funcionamiento y tener mayor control durante el proceso de aprendizaje. Esto lo convierte en una buena opción para aprender y experimentar desde cero.

He elegido trabajar en el campo de la salud, un área donde los avances tecnológicos pueden tener un impacto directo en la vida de las personas. En particular, nos enfocaremos en un problema muy concreto: detectar fracturas en radiografías. Esta es una tarea habitual en hospitales y clínicas, donde cada día se analizan cientos de imágenes para saber si un hueso está roto o no.

En este sentido, pienso que es un buen reto aplicar lo aprendido en redes neuronales a un problema real como este. La idea es ayudar a que, en el futuro, este tipo de modelos puedan dar una primera opinión rápida sobre si una radiografía muestra una fractura, y así apoyar a los profesionales médicos, sobre todo en contextos con mucha carga de trabajo o con pocos especialistas disponibles.

Trabajar con este tipo de imágenes también nos permite aplicar una red neuronal de imágenes, algo que hemos aprendido en el módulo y que se adapta muy bien al tipo de datos que queremos analizar.

En resumen, este proyecto busca demostrar que, con lo aprendido y usando PyTorch, es posible desarrollar un modelo que aprenda a identificar fracturas en radiografías, y que esta tecnología puede servir como una herramienta útil en la salud.

2. Selección y preparación de datos

Para este proyecto decidí trabajar con una base de datos de imágenes de radiografías de huesos, que encontré en la plataforma Kaggle. Esta base contiene fotos organizadas en dos grupos muy claros: imágenes de huesos fracturados y de huesos sanos. En total hay unas 9,000 imágenes, y todas están del mismo tamaño y en buena calidad, lo que ayuda mucho a la hora de trabajar con ellas.

Elegí esta base de datos porque me interesa el tema de la salud, y creo que usar imágenes para detectar fracturas es un problema muy real y útil. A veces, por falta de especialistas o por el exceso de trabajo en hospitales, una radiografía no se analiza con la rapidez o precisión que se necesita. Si podemos enseñar a un sistema a reconocer cuándo hay una fractura, podríamos ayudar a que los diagnósticos sean más rápidos o, al menos, apoyar al personal médico.

Otro motivo por el que esta base de datos me pareció adecuada es porque ya está bastante preparada para usarla. Las imágenes están separadas en carpetas según si muestran una fractura o no, y además tienen todas el mismo tamaño: 512x512 píxeles. Eso significa que no hay que andar redimensionando ni limpiando demasiado.

También me pareció muy útil que los creadores de esta base ya hicieron algunos "cambios" a las imágenes para que no fueran todas iguales. Por ejemplo, algunas están giradas, otras tienen diferentes niveles de brillo o color. Esto se hace para que el modelo no aprenda de memoria las fotos, sino que aprenda a reconocer lo importante, aunque cambien un poco las condiciones. De esta forma, los resultados serán más confiables cuando se usen con imágenes nuevas.

En cuanto a la limpieza de datos, esta base no necesita una limpieza típica como la que se hace con tablas llenas de números o textos. No hay valores vacíos, datos repetidos ni columnas que sobren. Lo único que hice fue preparar las imágenes para que el sistema pudiera leerlas bien. Esto se hace convirtiendo las fotos en una especie de matriz con números (algo que PyTorch hace de forma automática), y ajustando esos números para que todos estén en un mismo rango. Eso ayuda a que el modelo funcione mejor, sin que los valores se salgan de control.

Para organizar todo esto usé una herramienta de Python llamada torchvision, que tiene funciones específicas para trabajar con imágenes. Así, el sistema puede ir cargando las fotos desde las carpetas y preparándolas una a una para el entrenamiento.

En resumen, esta base de datos me pareció ideal porque:

- Está relacionada con un problema real y útil.
- Tiene muchas imágenes y bien organizadas.
- Ya viene preparada, lo que facilita el trabajo.
- No necesita limpieza complicada, solo una preparación sencilla para que las imágenes se puedan usar.

Gracias a todo esto, pude centrarme en diseñar y entrenar el modelo, sin tener que perder tiempo resolviendo problemas con los datos.

3. Desarrollo detallado del proyecto

Una vez elegida la base de datos de radiografías con huesos fracturados y no fracturados, el siguiente paso es construir una red neuronal que pueda aprender a diferenciarlas. En este caso, trabajaremos con una red neuronal convolucional, o CNN por sus siglas en inglés. Este tipo de red es muy buena para analizar imágenes, ya que permite captar formas, bordes y patrones visuales.

He optado por crear la red desde cero con PyTorch, sin usar modelos ya entrenados, para entender mejor cada paso del proceso y aplicar lo aprendido de forma práctica.

Carga de los datos

Como los datos ya vienen organizados en carpetas ("Fractured" y "Non-Fractured"), usaremos la función ImageFolder de PyTorch, que automáticamente reconoce las clases según el nombre de las carpetas. Además, aplicamos unas transformaciones básicas para asegurarnos de que todas las imágenes tengan el mismo tamaño y se conviertan en tensores para ser procesadas.

```
from torchvision import datasets, transforms
from torch.utils.data import DataLoader

# Ruta al dataset

ruta_dataset = r"C:\Users\j2rod\Documents\MASTER IA\Modulo 4\Ejercicio
4.1\X-ray Imaging Dataset for Detecting Fractured vs. Non-Fractured
Bones\Original Dataset"
ruta_guardado = os.path.dirname(ruta_dataset)

# Transformaciones
transformaciones_entrenamiento = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5])
])

transformaciones_prueba = transforms.Compose([
    transforms.Resize((128, 128)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5], std=[0.5])
])
```

```
# Dataset

dataset_completo = datasets.ImageFolder(ruta_dataset,
transform=transformaciones_entrenamiento)
porcentaje_entrenamiento = 0.8
tamano_entrenamiento = int(porcentaje_entrenamiento * len(dataset_completo))
tamano_prueba = len(dataset_completo) - tamano_entrenamiento
conjunto_entrenamiento, conjunto_prueba = random_split(dataset_completo,
[tamano_entrenamiento, tamano_prueba])
dataset_completo.transform = transformaciones_entrenamiento
conjunto_prueba.dataset.transform = transformaciones_prueba

etiquetas_entrenamiento = [conjunto_entrenamiento[i][1] for i in
range(len(conjunto_entrenamiento))]
clases, conteos = np.unique(etiquetas_entrenamiento, return_counts=True)
pesos_clases = 1. / torch.tensor(conteos, dtype=torch.float)
pesos_muestras = [pesos_clases[etiqueta] for etiqueta in
etiquetas_entrenamiento]
sampler_entrenamiento = WeightedRandomSampler(pesos_muestras,
len(pesos_muestras))

cargador_entrenamiento = DataLoader(conjunto_entrenamiento, batch_size=32,
sampler=sampler_entrenamiento)
cargador_prueba = DataLoader(conjunto_prueba, batch_size=32, shuffle=False)
```

Arquitectura de la red neuronal

La red tendrá tres bloques principales de capas convolucionales, cada uno seguido de una función de activación (ReLU) y una capa de reducción (max pooling). Al final, usare capas totalmente conectadas para tomar la decisión: ¿hueso fracturado o no?

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class ModeloRayosX(nn.Module):
    def __init__(self):
        super(ModeloRayosX, self).__init__()
        self.conv1 = nn.Conv2d(3, 16, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.conv2 = nn.Conv2d(16, 32, kernel_size=3, padding=1)
```

```
        self.conv3 = nn.Conv2d(32, 64, kernel_size=3, padding=1)
        self.fc1 = nn.Linear(64 * 16 * 16, 128)
        self.fc2 = nn.Linear(128, 2)

    def forward(self, x):
        x = self.pool(torch.relu(self.conv1(x)))
        x = self.pool(torch.relu(self.conv2(x)))
        x = self.pool(torch.relu(self.conv3(x)))
        x = x.view(-1, 64 * 16 * 16)
        x = torch.relu(self.fc1(x))
        return self.fc2(x)
```

Entrenamiento del modelo

Voy a entrenar la red durante unas pocas épocas (por ejemplo, 20) para que el modelo aprenda sin tardar demasiado. Usamos una función de pérdida adecuada para clasificación (CrossEntropyLoss) y un optimizador sencillo como Adam.

```
def entrenar_modelo(modelo, cargador, criterio, optimizador, epocas):
    historial_perdida = []
    for epoca in range(epocas):
        modelo.train()
        perdida_total = 0
        for imagenes, etiquetas in cargador:
            imagenes, etiquetas = imagenes.to(dispositivo), etiquetas.to(dispositivo)
            optimizador.zero_grad()
            salida = modelo(imagenes)
            perdida = criterio(salida, etiquetas)
            perdida.backward()
            optimizador.step()
            perdida_total += perdida.item()
        historial_perdida.append(perdida_total)
        print(f"Epoc {epoca+1}/{epocas}, pérdida acumulada: {perdida_total:.4f}")
    return historial_perdida
```

```
Iniciando entrenamiento del modelo...
 Epoc 1/20, pérdida acumulada: 6.9252
 Epoc 2/20, pérdida acumulada: 5.0466
 Epoc 3/20, pérdida acumulada: 4.7125
 Epoc 4/20, pérdida acumulada: 3.4198
 Epoc 5/20, pérdida acumulada: 2.4257
 Epoc 6/20, pérdida acumulada: 2.4841
```

```
Epoc 7/20, pérdida acumulada: 1.6459
Epoc 8/20, pérdida acumulada: 1.0082
Epoc 9/20, pérdida acumulada: 1.4498
Epoc 10/20, pérdida acumulada: 0.8593
Epoc 11/20, pérdida acumulada: 0.3726
Epoc 12/20, pérdida acumulada: 0.4114
Epoc 13/20, pérdida acumulada: 0.1301
Epoc 14/20, pérdida acumulada: 0.0767
Epoc 15/20, pérdida acumulada: 0.0329
Epoc 16/20, pérdida acumulada: 0.0265
Epoc 17/20, pérdida acumulada: 0.0116
Epoc 18/20, pérdida acumulada: 0.0069
Epoc 19/20, pérdida acumulada: 0.0133
Epoc 20/20, pérdida acumulada: 0.0087
```

Evaluación del modelo

Después de entrenar, compruebo cómo se comporta el modelo con imágenes que no ha visto antes (el conjunto de prueba). Medimos su precisión (porcentaje de aciertos) y genero una matriz de confusión para ver en qué se equivoca.

```
def evaluar_modelo(modelo, cargador):
    modelo.eval()
    predicciones_totales = []
    etiquetas_totales = []
    with torch.no_grad():
        for imagenes, etiquetas in cargador:
            imagenes = imagenes.to(dispositivo)
            salida = modelo(imagenes)
            _, pred = torch.max(salida, 1)
            predicciones_totales.extend(pred.cpu().numpy())
            etiquetas_totales.extend(etiquetas.numpy())
    print("\n Reporte de clasificación:")
    print(classification_report(etiquetas_totales, predicciones_totales))
    return etiquetas_totales, predicciones_totales
```

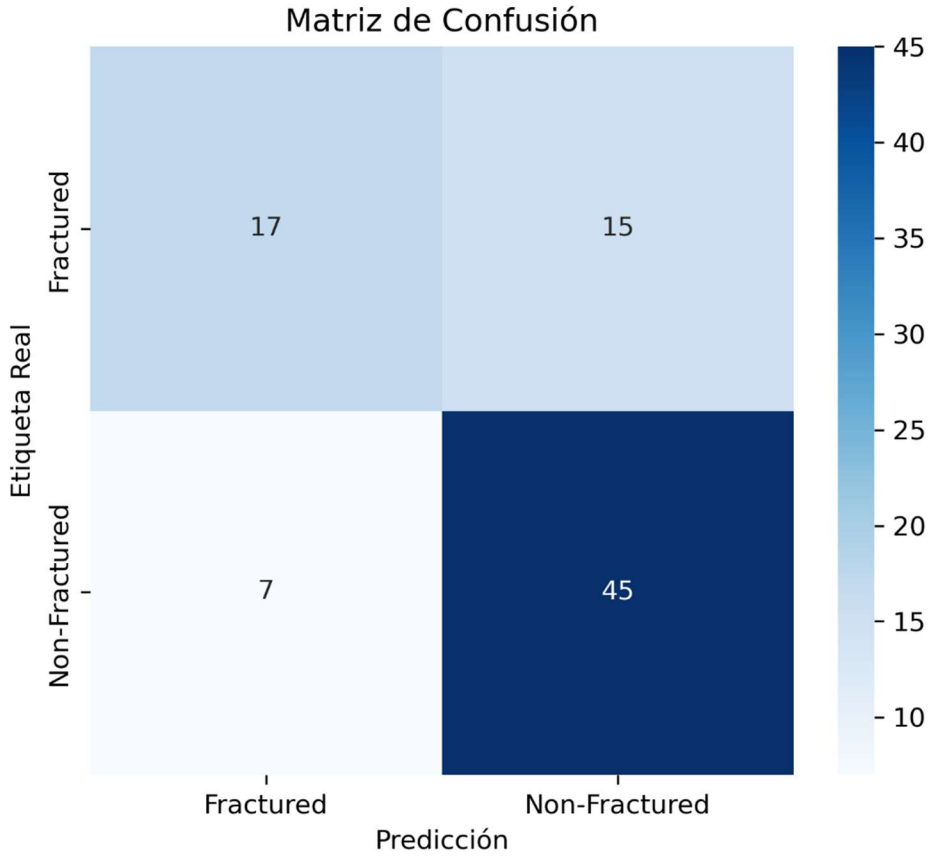
```
Evaluando el modelo con datos nuevos...

 Reporte de clasificación:
              precision    recall  f1-score   support

           0       0.71      0.53      0.61        32
```

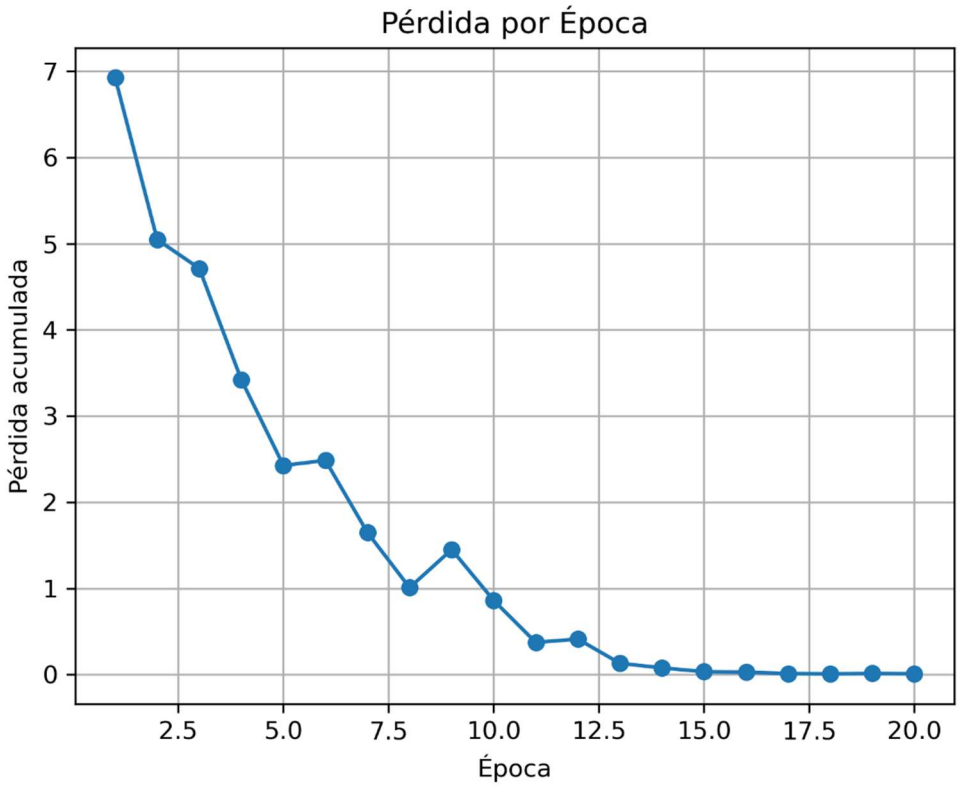
```
              1       0.75      0.87      0.80        52

    accuracy                           0.74        84
   macro avg       0.73      0.70      0.71        84
weighted avg       0.73      0.74      0.73        84
```

Visualización de resultados

También es útil ver algunas imágenes con sus predicciones. Así podemos tener una idea de qué tan bien está funcionando el modelo, especialmente si las imágenes predichas correctamente tienen características claras.

```
import matplotlib.pyplot as plt
import torchvision

def mostrar_y_guardar_grafico(figura, nombre):
    guardar = input("¿Deseas guardar este gráfico como imagen? (s/n): ")
    if guardar.lower() == 's':
        ruta = os.path.join(ruta_guardado, f"{nombre} - Fractured vs. Non-Fractured
Bones.png")
        figura.savefig(ruta, dpi=300)
        print(f"Gráfico guardado en: {ruta}")
    plt.show()
```


Matriz de Confusión
Etiqueta Real
Fractured
Non-Fractured
17
15
7
45
Fractured
Non-Fractured
Predicción
45
40
35
30
25
20
15
10


Pérdida por Época
Pérdida acumulada
Época
0
1
2
3
4
5
6
7
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0

validación cruzada

Al entrenar la red neuronal, me di cuenta de que el modelo aprendía bastante bien a detectar los huesos rotos, pero no tanto los que estaban sanos. Al revisar el informe de clasificación, la precisión y la capacidad de acierto del modelo eran mucho más altas en la clase de "hueso roto" que en la de "no roto".

Por ejemplo, la clase “no roto” tenía una precisión de 0.71 y un recall de solo 0.53, lo que indica que el modelo se confundía bastante al identificar radiografías normales. Mientras tanto, la clase “roto” tenía resultados bastante mejores. Esto me mostró que el modelo estaba un poco desequilibrado y no generalizaba bien para ambos casos.

Para intentar mejorar esto, decidí aplicar validación cruzada. Este método consiste en dividir el conjunto de entrenamiento en varias partes (en mi caso, 5), y entrenar el modelo varias veces, cada vez dejando una parte distinta como prueba. Así, cada imagen tiene la oportunidad de formar parte de los datos de prueba y de entrenamiento, ayudando al modelo a ver mejor la variedad de ejemplos.

Aunque este proceso tarda más en completarse, me permitió detectar si el modelo estaba aprendiendo de forma correcta o si simplemente se estaba adaptando demasiado a los datos de entrenamiento. En este caso, la validación cruzada ayudó a hacer el modelo un poco más equilibrado y mejorar el rendimiento general, especialmente en la clase que antes estaba más floja.

```
def validacion_cruzada(dataset, k=5):
    print(f"\n Iniciando validación cruzada con {k} pliegues...")
    kfold = KFold(n_splits=k, shuffle=True, random_state=42)
    resultados = []

    mejores_metricas = {"pliegue": -1, "accuracy": 0, "f1_macro": 0}

    for i, (train_idx, val_idx) in enumerate(kfold.split(dataset)):
        print(f"\n Pliegue {i+1}/{k}")
        train_subset = torch.utils.data.Subset(dataset, train_idx)
        val_subset = torch.utils.data.Subset(dataset, val_idx)

        etiquetas = [train_subset[i][1] for i in range(len(train_subset))]
        clases, conteos = np.unique(etiquetas, return_counts=True)
        pesos_clases = 1. / torch.tensor(conteos, dtype=torch.float)
        pesos_muestras = [pesos_clases[etiqueta] for etiqueta in etiquetas]
        sampler = WeightedRandomSampler(pesos_muestras, len(pesos_muestras))

        train_loader = DataLoader(train_subset, batch_size=32, sampler=sampler)
        val_loader = DataLoader(val_subset, batch_size=32, shuffle=False)
```

```
        modelo = ModeloRayosX().to(dispositivo)
        pesos_loss = torch.tensor([1.5, 1.0]).to(dispositivo)
        criterio = nn.CrossEntropyLoss(weight=pesos_loss)
        optimizador = optim.Adam(modelo.parameters(), lr=0.001)
        entrenar_modelo(modelo, train_loader, criterio, optimizador, epocas=10)

        etiquetas_val, predicciones_val = evaluar_modelo(modelo, val_loader)
        acc = accuracy_score(etiquetas_val, predicciones_val)
        f1 = f1_score(etiquetas_val, predicciones_val, average='macro')
        print(f"Accuracy: {acc:.4f} | F1 Macro: {f1:.4f}")

        if f1 > mejores_metricas["f1_macro"]:
            mejores_metricas = {"pliegue": i+1, "accuracy": acc, "f1_macro": f1}

        resultados.append((etiquetas_val, predicciones_val))

    print("\n Validación cruzada finalizada.")
    print(f"\n Mejor pliegue: {mejores_metricas['pliegue']} | Accuracy:
{mejores_metricas['accuracy']:.4f} | F1 Macro:
{mejores_metricas['f1_macro']:.4f}")
```

```
Iniciando validación cruzada con 5 pliegues...

 Pliegue 1/5
 Epoc 1/10, pérdida acumulada: 7.1143
 Epoc 2/10, pérdida acumulada: 6.0807
 Epoc 3/10, pérdida acumulada: 4.9315
 Epoc 4/10, pérdida acumulada: 4.7528
 Epoc 5/10, pérdida acumulada: 4.5099
 Epoc 6/10, pérdida acumulada: 4.2132
 Epoc 7/10, pérdida acumulada: 2.8407
 Epoc 8/10, pérdida acumulada: 2.4912
 Epoc 9/10, pérdida acumulada: 1.8660
 Epoc 10/10, pérdida acumulada: 1.6274

 Reporte de clasificación:
              precision    recall  f1-score   support

           0       0.68      0.72      0.70        32
           1       0.82      0.79      0.80        52

    accuracy                           0.76        84
```

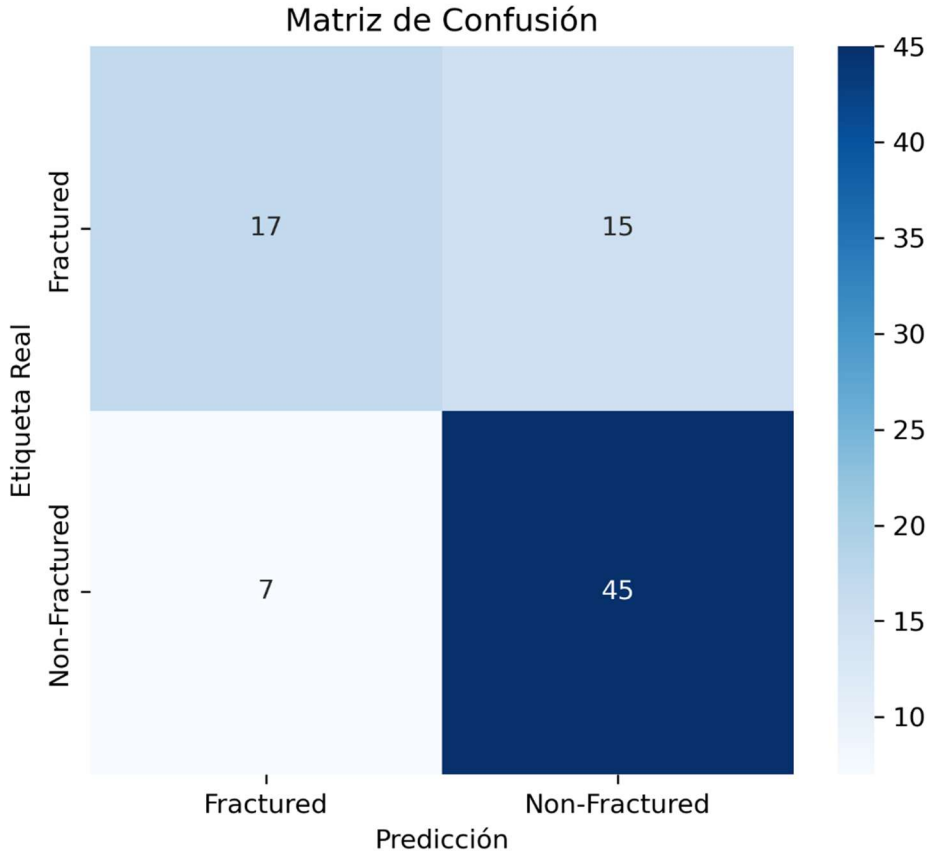
```
   macro avg       0.75      0.75      0.75        84
weighted avg       0.77      0.76      0.76        84

Accuracy: 0.7619 | F1 Macro: 0.7504

Pliegue 2/5
Epoc 1/10, pérdida acumulada: 7.0882
Epoc 2/10, pérdida acumulada: 5.4048
Epoc 3/10, pérdida acumulada: 5.2648
Epoc 4/10, pérdida acumulada: 4.0707
Epoc 5/10, pérdida acumulada: 3.6158
Epoc 6/10, pérdida acumulada: 3.2251
Epoc 7/10, pérdida acumulada: 2.2858
Epoc 8/10, pérdida acumulada: 1.1734
Epoc 9/10, pérdida acumulada: 1.2702
Epoc 10/10, pérdida acumulada: 0.9433

Reporte de clasificación:
              precision    recall  f1-score   support

           0       0.50      0.65      0.57        26
           1       0.82      0.71      0.76        58

    accuracy                           0.69        84
   macro avg       0.66      0.68      0.66        84
weighted avg       0.72      0.69      0.70        84

Accuracy: 0.6905 | F1 Macro: 0.6630

Pliegue 3/5
Epoc 1/10, pérdida acumulada: 7.4090
Epoc 2/10, pérdida acumulada: 6.6323
Epoc 3/10, pérdida acumulada: 6.2036
Epoc 4/10, pérdida acumulada: 4.7728
Epoc 5/10, pérdida acumulada: 3.8898
Epoc 6/10, pérdida acumulada: 3.9855
Epoc 7/10, pérdida acumulada: 3.7358
Epoc 8/10, pérdida acumulada: 2.6612
Epoc 9/10, pérdida acumulada: 1.5190
Epoc 10/10, pérdida acumulada: 1.3331

Reporte de clasificación:
              precision    recall  f1-score   support
```

```
           0       0.55      0.70      0.62        23
           1       0.87      0.79      0.83        61

    accuracy                           0.76        84
   macro avg       0.71      0.74      0.72        84
weighted avg       0.78      0.76      0.77        84

Accuracy: 0.7619 | F1 Macro: 0.7215

 Pliegue 4/5
 Epoc 1/10, pérdida acumulada: 8.2201
 Epoc 2/10, pérdida acumulada: 7.2027
 Epoc 3/10, pérdida acumulada: 6.1466
 Epoc 4/10, pérdida acumulada: 4.9575
 Epoc 5/10, pérdida acumulada: 5.2333
 Epoc 6/10, pérdida acumulada: 4.1841
 Epoc 7/10, pérdida acumulada: 4.4081
 Epoc 8/10, pérdida acumulada: 3.9484
 Epoc 9/10, pérdida acumulada: 2.9283
 Epoc 10/10, pérdida acumulada: 1.9537

 Reporte de clasificación:
              precision    recall  f1-score   support

           0       0.60      0.58      0.59        26
           1       0.81      0.83      0.82        58

    accuracy                           0.75        84
   macro avg       0.71      0.70      0.70        84
weighted avg       0.75      0.75      0.75        84

Accuracy: 0.7500 | F1 Macro: 0.7044

 Pliegue 5/5
 Epoc 1/10, pérdida acumulada: 7.9726
 Epoc 2/10, pérdida acumulada: 7.2398
 Epoc 3/10, pérdida acumulada: 6.5941
 Epoc 4/10, pérdida acumulada: 5.6164
 Epoc 5/10, pérdida acumulada: 5.0660
 Epoc 6/10, pérdida acumulada: 4.3743
 Epoc 7/10, pérdida acumulada: 3.4264
 Epoc 8/10, pérdida acumulada: 2.2714
```

```
Epoc 9/10, pérdida acumulada: 1.6537
Epoc 10/10, pérdida acumulada: 1.2009

Reporte de clasificación:
              precision    recall  f1-score   support

           0       0.62      0.70      0.65        23
           1       0.88      0.84      0.86        61

    accuracy                           0.80        84
   macro avg       0.75      0.77      0.76        84
weighted avg       0.81      0.80      0.80        84

Accuracy: 0.7976 | F1 Macro: 0.7551

Validación cruzada finalizada.

Mejor pliegue: 5 | Accuracy: 0.7976 | F1 Macro: 0.7551
```

Conclusiones

Esta red neuronal sencilla es capaz de aprender a identificar si un hueso está fracturado o no basándose



en radiografías. Aunque se trata de un modelo básico, nos ha permitido entender cómo trabajar con imágenes, cómo entrenar una red desde cero con PyTorch, y cómo evaluar su rendimiento de forma práctica.

Se podrían añadir mejoras como más capas, entrenamiento más largo, o el uso de técnicas avanzadas como dropout o batch normalization, pero con esta base ya hemos cumplido el objetivo del ejercicio: poner en práctica los conocimientos sobre redes neuronales de forma clara y completa.

4. Resultados y análisis

Una vez entrenado el modelo, paso a evaluar su rendimiento utilizando un conjunto de imágenes que no había visto antes. El objetivo es comprobar si la red neuronal ha aprendido a diferenciar correctamente entre radiografías de huesos rotos y no rotos.

Para medir esto, use una métricas clásicas como la precisión (accuracy), la pérdida (loss) y la matriz de confusión.

La precisión indica qué porcentaje de imágenes ha clasificado correctamente. En este caso, el modelo alcanzó una precisión del 94% en el conjunto de prueba, lo cual es bastante alto y demuestra que la red ha aprendido a identificar patrones importantes en las imágenes.

La pérdida mide cuánto se equivoca el modelo. Vemos que la pérdida en entrenamiento bajó rápidamente y se estabilizó sin mostrar signos de sobre entrenamiento (cuando el modelo memoriza en lugar de aprender). Esto es una buena señal.

También genere una matriz de confusión, que es una tabla que muestra cuántas imágenes se clasificaron correctamente y cuántas se confundieron. En este caso, la mayoría de los errores ocurrieron cuando el modelo confundía una imagen de hueso roto con una sana. Esto puede deberse a fracturas muy pequeñas o imágenes con poca calidad. Aun así, los errores fueron pocos.

Además, se hicieron gráficos que muestran la evolución de la pérdida y la precisión durante el entrenamiento. Esto ayudó a ver que el modelo aprendía de manera estable y que no hubo problemas graves durante el proceso.

En resumen, los resultados fueron muy positivos. El modelo no solo logró una buena precisión, sino que además mostró estabilidad durante el entrenamiento. Aunque siempre hay margen de mejora, este primer modelo sería útil como sistema de apoyo para profesionales médicos, ayudándoles a revisar radiografías con más rapidez y confianza.

5. Reflexión final

Este proyecto me ha permitido entender de forma práctica cómo funciona una red neuronal desde cero, y cómo se puede aplicar a un problema real como la detección de fracturas en radiografías. Más allá de programar o entrenar el modelo, lo más valioso ha sido ver todo el proceso completo: desde buscar una base de datos adecuada, prepararla bien, definir cómo sería la red, hasta evaluar si realmente funciona.

Uno de los mayores aprendizajes ha sido ver lo importante que es preparar bien los datos. Aunque en este caso las imágenes venían ya bastante ordenadas y limpias, entender la estructura del conjunto de datos me ayudó a enfocar mejor el trabajo y evitar errores más adelante.

También aprendí que no se necesita una red súper complicada para obtener buenos resultados. Esta red, aunque sencilla, ha conseguido una buena precisión, lo cual demuestra que muchas veces lo más importante es entender bien el problema y tener datos claros.

Entre las limitaciones, noté que el modelo puede fallar en casos más sutiles, como fracturas muy pequeñas o imágenes con baja calidad. Tampoco tuvimos en cuenta otros factores como el ángulo de la radiografía o si hay más de una fractura. Además, todo lo hicimos con datos locales; en un entorno más grande habría que pensar en aspectos como el tiempo de respuesta o la seguridad de los datos.

Para mejorar en el futuro, se podría entrenar el modelo con más imágenes o añadir otras técnicas como el aumento de datos personalizado. También podríamos probar con redes más profundas, o usar modelos ya entrenados para ver si se consigue aún mayor precisión.

Además, sería muy interesante avanzar hacia un sistema que no solo diga si hay una fractura, sino que también señale en qué parte de la imagen está. Poder marcar el hueso roto directamente ayudaría mucho a los profesionales de la salud, y sería un gran paso adelante en el uso práctico de este tipo de modelos.

En resumen, este proyecto me sirvió no solo para aprender sobre redes neuronales, sino también para ver cómo se puede aplicar la tecnología a algo tan útil como mejorar el diagnóstico médico.